HireAssist

Architecture Decision Records

Software Architecture term project · Chulalongkorn University

Patiphon Puntusin · Thanabul Parodom · Thanwarat Korcharoenkiat · Rerngrit Jangsri

1 October 2026

Contents

ID	Decision	Status
ADR-001	Capability-aligned service decomposition behind an API gateway	Accepted
ADR-002	One independently retried unit of work per resume, held in a claimable work table	Accepted
ADR-003	PostgreSQL as system of record, MongoDB for AI-derived documents	Accepted
ADR-004	All model access through one AI Service, on a managed API that does not train on our data	Accepted
ADR-005	Each service chooses its own language and framework, within shared contracts	Proposed
ADR-006	One deployment per customer company	Accepted
ADR-007	gRPC on the AI Service boundary	Accepted
ADR-008	Turning a resume into a candidate profile is a model call	Accepted

These records are a stack. Each states what was decided on its date and nothing after it; an earlier record is never edited because of a later one. To find the current position on a question, read forward from the record that first raised it.

What later records have changed

Because an earlier record is never edited, this is the only place that says which parts of an earlier one no longer hold. Read it before acting on any record below.

Record	Still in force	Changed by a later record
ADR-001	The five services, the capability boundaries, the gateway, Kubernetes discovery, and REST on every boundary **except the AI Service**.	The single-backend-language assumption is withdrawn (ADR-005). The workspace concept is gone (ADR-006): the isolation force in *Issue*, the gateway's workspace resolution, the internal-call cost, and the *Identity & Workspace* name. The AI Service boundary is gRPC, not REST (ADR-007); the rule stands everywhere else. Its argument that Resume Processing and the AI Service have opposite resource profiles no longer holds — both now wait on a model call (ADR-008); the boundary stands on failure isolation and credential containment.
ADR-002	All of it.	—
ADR-003	The PostgreSQL / MongoDB split and the rule that decides which store new data goes in.	Documents carry `candidate_id` alone, with no `workspace_id` (ADR-006).
ADR-004	The single credential, no fallback model, the service being unreachable from the gateway, and the rule that it never receives a raw file.	Those operations are carried over gRPC rather than REST (ADR-007). There are now four, not three: profile extraction is a model call served here (ADR-008), and it is the one operation that receives unstructured resume text, which narrows the minimisation rule and moves the protected-attribute stripping into it.
ADR-005	The principle and the guardrails. Still *Proposed* — languages await service ownership.	The worst case its access-check argument guards against is now a privilege mistake inside one company, not a cross-company leak (ADR-006). Its contract guardrail now covers two mechanisms: OpenAPI for REST boundaries, a `.proto` for the AI Service (ADR-007). Its Python preference for Resume Processing now rests on PDF text extraction alone — structuring the text is no longer done there (ADR-008).
ADR-006	All of it.	—
ADR-007	All of it.	—
ADR-008	All of it.	—

ADR-001: Capability-aligned service decomposition behind an API gateway

Date: 2026-09-11 **Deciders:** Patiphon Puntusin, Thanabul Parodom, Thanwarat Korcharoenkiat, Rerngrit Jangsri

> *In the context of building HireAssist as microservices, facing three workloads that differ in who triggers them, how long they take and what data they own, we decided to split the system into five services behind one API gateway — boundaries drawn by capability, not by technical layer — to achieve independent scaling of screening and one enforcement point for identity, accepting that data ownership is now distributed and that erasure and reporting must cross service boundaries.*

Summary

Issue

Nothing else can be designed until the boundaries exist: the datastores, the model integration and the way work is handed between services all sit on them. Four forces shape the split.

Batch screening contains two workloads that do not belong in one process. Accepting a batch of resumes must return in milliseconds. Screening one of them is parsing plus model calls — seconds normally, minutes for a long file, never when the provider is down. Share a process and the recruiter-facing API inherits the latency and availability of the slowest external dependency in the system.

Retention enforcement is clock-driven and destructive. It deletes candidate data wherever it lives and must prove it did. That is a different responsibility from serving a screen, it runs when nobody is watching, and it needs an owner: a privacy obligation with no clear home is one nobody implements.

Every request must carry a verified identity and role. A rule enforced in five places is a rule enforced in four places plus a bug.

The team is four students learning this stack while building it. Every service costs a deployment, a pipeline, a schema and a place for a bug to hide. Over-decomposition is the most likely way this project fails to ship.

Decision

Five services and one gateway, aligned to capabilities:

Service	Owns
API Gateway	No domain data. Terminates TLS, validates the session token, resolves workspace and role, rate-limits, routes.
Identity & Workspace Service	Accounts, workspaces, memberships, roles, sessions
Hiring Service	Job openings, criteria, screening batches, decisions, interview guides
Resume Processing Service	Candidate profiles, parsed resume text
AI Service	Prompts and the model credential — no domain data
Compliance & Insights Service	Retention policy, pipeline metrics, audit log

Every boundary is REST over HTTP/JSON — one protocol, no exceptions. Client to gateway, gateway to service, and service to service all speak the same thing. It is browser-facing and public at the edge, readable in a network tab, debuggable without a client toolchain, and it means four students learn one wire format instead of three.

Two boundaries would each have argued for something else, and are deliberately served by REST anyway:

- **Hiring and Resume Processing → AI Service** is a high-volume internal call whose contract must not drift: *(profile, criteria) → (score, must-have check, justification)*. It is a JSON request and a JSON response, held stable by an OpenAPI document rather than by a generated stub.
- **Hiring → Resume Processing** is long-running work, not a query. REST cannot make it synchronous: Hiring hands over one resume and gets an immediate acknowledgement, and Resume Processing calls back with the result when it has one. Screening remains asynchronous; what carries the asynchrony is a separate decision.

Service discovery is Kubernetes. Each service is a Deployment behind a ClusterIP Service; services address each other by in-cluster DNS (`ai-service.hireassist.svc`) and kube-proxy balances across healthy Pods. Readiness probes decide membership, so discovery and health checking are one mechanism rather than two that can disagree. No separate registry is deployed.

Status

Accepted

Group

Decomposition · Communication

Details

Assumptions

- Go is the backend language for all services. One language keeps four students able to read each other's code.
- Deployment is to Kubernetes — managed, or local for the demo. Worth re-checking early: if the team cannot get a cluster running, discovery must be reconsidered, because everything depends on it.
- Load is one company's hiring: tens of concurrent recruiters, batches of 20–200 resumes. Nothing here is sized for internet-scale traffic.
- The same four people operate every service. There is no platform team.

Constraints

- An API gateway and a documented discovery approach are required of us, and so, eventually, is a spread of communication styles — at least one service reached by REST, one by RPC and one driven by a message broker. **This record knowingly does not satisfy the second half.** We are building the boundaries first and the protocol variety second, on the argument that a boundary in the wrong place is expensive to move and a protocol is not. The obligation is deferred, not denied.

- Privacy law makes the *location* of personal data a design concern: the fewer services holding resume content, the smaller the surface that retention, access control and breach response must cover.

Positions

1. **Modular monolith plus one background worker.** Two deployables, one schema, no network between the parts. The least work, and defensible in production at this size.
2. **One service per use case** — eight services.
3. **Decomposition by technical layer** — API, parsing, scoring, persistence.
4. **The chosen split with the AI Service merged into Resume Processing** — four services.
5. **The chosen split with compliance folded into Hiring**, run by a scheduler inside it.
6. **A separate service registry** such as Consul, instead of Kubernetes DNS.
7. **A protocol chosen per boundary** — REST at the edge, gRPC into the AI Service, a message broker between Hiring and Resume Processing — instead of REST everywhere.

Argument

Against the monolith (1). It puts the recruiter-facing API in the same process as work that blocks on an external model, so a provider outage degrades sign-in and the dashboard, which do not depend on the model at all. It also gives one worker pool to two workloads with opposite resource profiles: parsing is CPU-bound and finishes in seconds, while scoring waits on a remote call — cheap in CPU, long in wall-clock, bounded by the provider's rate limit rather than our hardware. One pool must be sized for one or the other, which makes throughput hard to scale and harder to demonstrate.

Against one service per use case (2). Creating a job, screening against it and generating interview questions are three views of the same two aggregates — the job opening and the candidate profile. Splitting them puts a network call and an eventual-consistency problem between "the criteria" and "the score against the criteria" and buys nothing: they deploy together, change together, scale together. Eight pipelines, eight schemas, hypothetical benefit.

Against layered services (3). A layer is not a capability. Adding one field to a criterion would touch the API, scoring and persistence services — every change a coordinated three-service release. This is the split that looks organised on a diagram and behaves worst in practice.

Against merging the AI Service (4). Extracting criteria from a paragraph and writing an interview guide are interactive; neither is batch work. Merging would route interactive calls through a batch worker, or duplicate prompt and credential handling in two codebases. One service for every model call is also what makes a single point of control possible: one place holds the credential, caps spending, and changes when the provider changes.

Against folding compliance into Hiring (5). A retention sweep would compete with recruiter traffic in one process. More importantly, retention must be auditable in isolation: *which component may delete candidate data?* should have a one-word answer.

Against a separate registry (6). It means running and explaining a second discovery mechanism when the platform already provides one, and it puts registration in application code, where a service that crashes without deregistering leaves a stale entry. Kubernetes derives the same information from probes it already runs. The cost is that the team must learn Kubernetes.

Against a protocol per boundary (7). This is the one we expect to revisit, and it is the one we are most likely to be told is wrong. Each specialised choice is individually defensible: a generated stub cannot drift the way a hand-written JSON client can, and a broker gives retry, backpressure and dead-lettering for free rather than as code we write. What decided it is that none of those benefits arrives before the boundaries themselves are proven correct, and all of them arrive with a bill paid up front — a second wire format, a code-generation step in every build, broker infrastructure to run locally on four laptops, and three failure modes to learn instead of one.

The boundaries are the expensive thing to get wrong. A boundary in the wrong place is re-owned data, a re-cut schema and a coordinated release; a protocol in the wrong place is one adapter rewritten behind an interface that already exists. So we place the boundaries now, run them all over the protocol every one of us can already debug, and change a protocol when a boundary demonstrates it needs one — the AI call when contract drift actually bites, the screening handover when our own retry logic starts reimplementing a broker badly. The risk we accept is that "first" becomes "only", and that the day we do need a broker we will be retrofitting it into code that assumed a synchronous acknowledgement.

For the chosen split. Its three boundaries fall where three properties genuinely differ: *who triggers the work* — a human, a queued message, or the clock; *how long it may take* — milliseconds, minutes, or a full sweep; and *what it owns* — the transactional hiring record, the derived profile, or the retention policy and audit log. A boundary separating things that differ on all three will still be in the right place in six months.

Implications

What this buys us

- Screening scales by adding Resume Processing replicas without touching the recruiter-facing API. This is the quality attribute the project demonstrates.
- The gateway is the one place a request is bound to a workspace and a role, so isolation is structural rather than a convention repeated in five codebases.
- The AI Service is the entire blast radius of the model credential and the entire token cost centre, making the provider a configuration choice rather than a rewrite.
- Scoring stays a separable *(profile, criteria)* operation callable by anything — the property any future re-matching over the candidate archive depends on.

What it costs us

- **Retention must erase data it does not own.** The policy lives in one service, the data in two others, so erasure becomes a multi-service protocol with a verification step. "Deleted" becomes eventually consistent — precisely what a regulator asks about.
- **The dashboard cannot be a query.** Metrics span services, so they must be maintained incrementally from domain events. Numbers can lag, and a lost event means a permanently wrong count unless something reconciles.
- **No distributed transactions.** An accepted batch whose messages are never consumed is inconsistent state nothing detects on its own; a reconciliation sweep becomes a requirement.
- **Workspace identity must travel on every internal call.** If only the gateway checks, an internal caller with the wrong id silently reads another company's candidates. Every contract needs the field and every service must enforce it, not merely receive it.

- **The team must learn Kubernetes** — the largest schedule risk this decision creates, landing on all four members. Holding the protocol count at one is partly how we afford it.
- **Six deployables plus two databases is heavy for a laptop.** Local development needs resource limits or a reduced profile, or the demo machine becomes the constraint.
- **One protocol everywhere means the hard parts are ours to write.** REST gives us no retry, no backpressure, no delivery guarantee and no dead-letter path; wherever a boundary needs those, we build them by hand and test them ourselves. The first place that bites is the screening handover, and the honest risk is that we rebuild a broker slowly and badly instead of running one.
- **The contract between two services is a document, not a compiler error.** With no generated stubs, a field renamed on one side fails at runtime in the other. Contract tests have to do the work a code generator would otherwise have done for free.

Related

Related decisions

None precede this one. It is the first architectural decision recorded, and every later one inherits the structure it sets. It **opens**, without settling, four questions — each a decision in its own right:

- How work crosses the Hiring → Resume Processing boundary: what holds a resume between handover and result, the delivery guarantee, and retry.
- Where the scoring model runs and who provides it.
- What each side of these boundaries stores.
- What the services are written in, and how cross-cutting concerns are handled if that differs.

Related requirements

- **Sign-in and access management** (UC-0, UC-6) are served by the Identity & Workspace Service with authorisation at the gateway; **job creation, screening and interview questions** (UC-1–UC-3) by Hiring, Resume Processing and the AI Service; **pipeline monitoring and retention** (UC-4, UC-5) by Compliance & Insights.
- **Workspace scoping** — a requirement since withdrawn — and **FR-0.4**, role enforcement, both bound to the request at the gateway.
- **NFR-04, NFR-07** — a batch is acknowledged in seconds while screening scales by adding replicas; the second is the demonstrated quality attribute.
- **NFR-10** — no failure in the batch path may lose accepted work or take the interactive path down with it.
- **NFR-17** — the scoring model and prompt change inside the AI Service alone.
- **FR-5.3, FR-5.8** — erasure must reach every service holding candidate data, which this decomposition turns into a cross-service protocol.

Related artifacts

The architecture diagram and the Service–Operations–Collaborators table derive directly from this decomposition.

Related principles

- *The human still decides* — the gateway and Identity Service exist so the decision-maker is always an identified person.
- *Easily reversible* — merging two of these services later is a refactor; splitting a monolith later is a project.

Notes

The strongest objection was that a modular monolith is the better engineering choice at this size, and that we are decomposing because we were told to. Partly true, and worth recording: without that constraint we would likely have started with two deployables and split when a boundary hurt. What survives it is the *shape* of the split — these boundaries follow trigger, latency and ownership, not the requirement to use three protocols.

The Argument was rewritten after that objection to rest on the product's own evidence alone: fault isolation of the model path, and the opposite resource profiles of parsing and scoring. This record should stand if the constraint were removed. The dissent is kept because it was real and may be right.

We considered splitting the communication decision into its own record, then kept it here. With one protocol on every boundary there is no per-boundary rationale to separate out — the choice is a property of the decomposition, made once. If a boundary later takes a different protocol, *that* is its own record.

The service owning monitoring and retention is named **Compliance & Insights** rather than *Insights & Notifications* because retention ownership is its defining responsibility, and because notifications are not a service — each event's owner emits its own. A shared notification channel may exist later; it would be infrastructure, not a boundary.

ADR-002: One independently retried unit of work per resume, held in a claimable work table

Date: 2026-09-11 **Deciders:** Patiphon Puntusin, Thanabul Parodom, Thanwarat Korcharoenkiat, Rerngrit Jangsri

> *In the context of batch resume screening over a single wire protocol, facing per-file processing that takes seconds to minutes and depends on an external model that will fail, we decided to make each resume an independently claimable row in the screening service's own work table, handed over by an acknowledged REST call and answered by a REST callback, with bounded retry and a terminal failure state, to achieve throughput that rises with worker count and a batch that survives partial failure and a model outage without losing work, accepting that results become eventually consistent, that a resume may occasionally be processed — and paid for — twice, and that we are writing by hand what a message broker would have supplied.*

Summary

Issue

Batch screening is what the project exists for, and it cannot be built the obvious way.

A recruiter drops 20–200 resumes onto an open position. Each must be parsed out of a PDF, normalised into a profile, scored against weighted criteria, and given a written justification. Scoring is at least one model call: seconds when healthy, tens of seconds for a long resume, unbounded when the provider is rate-limiting. A batch of 200 is minutes of work in the good case. No HTTP request survives that, and no recruiter should watch a spinner while it happens.

Failure is the normal path, not the exceptional one. A corrupt file or a scanned image with no text layer; the scoring service unavailable; rate limits, timeouts, truncated responses. In a batch of 200, something failing is close to certain.

That makes the unit of failure the sharp question. A recruiter who uploads 200 resumes and gets nothing because file 147 was a scanned image has been failed by the system. Getting 199 results and one entry marked *needs manual review* is not a degraded outcome — it is the correct one.

Every crossing between services is REST, decided when the boundaries were drawn. So whatever holds a resume between "accepted" and "scored" has to live inside a service, not between them. This record is about what that thing is.

This is also where the demonstrated quality attribute lives: throughput must rise in proportion as workers are added, and no accepted resume may ever be lost. Both claims are falsifiable here, and the second keeps the first honest.

Decision

Each resume is one row of work. The batch is a count, not a unit of work.

8. **Accept and return.** Hiring validates the upload, rejecting anything that is not a PDF, stores the files, creates a batch with one `pending` entry per resume, and responds `202 Accepted` with the batch ID within two seconds regardless of batch size.
9. **Hand each resume over individually.** Hiring calls Resume Processing once per resume — *(batch id, resume id, file reference, criteria snapshot, callback address)*. The call returns as soon as the work is durably recorded; it does not wait for a score. Handover is retried by Hiring until acknowledged, so an entry is never left with no owner.
10. **The work table is the queue.** Resume Processing writes each handover as a row in its own PostgreSQL schema, holding the request, its state, its attempt count and its next-eligible time. Nothing else stores in-flight screening work, and no other service reads this table.
11. **Consume in parallel.** N replicas of Resume Processing poll their own table, claiming a bounded number of eligible rows at a time with `SELECT … FOR UPDATE SKIP LOCKED` so no two workers take the same resume. One row is one resume: parse → score → persist → call back. Concurrency changes by changing N, and nothing else.
12. **Guarantee delivery, not exactly-once.** A claim is a row-level lock, not a removal. A worker that crashes mid-resume releases its lock with the row still unfinished, so another worker picks it up — at-least-once, deliberately. The same holds for the callback: it is retried with backoff until Hiring acknowledges it.
13. **Make redelivery harmless.** Results are upserted on `(batch_id, resume_id)` in both services, so a re-processed resume or a repeated callback overwrites its own earlier result rather than creating a second one.
14. **Classify failures before retrying.** A *permanent* failure — a corrupt file, a scanned image with no text layer — is never retried; the row goes to `needs_manual_review` with the reason. A *transient* failure — 429, 5xx, a broken connection, or a scoring attempt past its 60-second bound — is retried a bounded number of times, the row's next-eligible time pushed out by exponential backoff with jitter.
15. **Park what does not recover.** Once the retry budget is spent the row moves to a terminal `failed` state with its last error and the entry is marked `needs_manual_review`. Failed rows stay in the table and are listable and replayable by an operator; nothing is silently scored zero and nothing is deleted.
16. **Completion is computed.** A batch is complete when every entry is terminal — `scored` or `needs_manual_review` — and reaching that state triggers the batch-finished notification. There is no all-or-nothing outcome. A periodic sweep re-arms rows left claimed past a lease bound and re-sends handovers for entries Hiring still shows as `pending`, which is the defence against work that vanishes on either side of the boundary.
17. **Stream results to the screen.** Each callback updates the shortlist view over server-sent events, so the ranking fills in as results land.

There is no message broker. Durability comes from a database transaction, ordering does not matter, and the retry, backoff and terminal-failure behaviour above is application code in one service.

Status

Accepted

Group

Communication · Scalability · Reliability

Details

Assumptions

- A batch is 20–200 resumes, each file at most 10 MB and usually far smaller. Nothing here assumes tens of thousands of files.
- Every service-to-service call is REST over HTTP/JSON, and no broker is deployed.
- The model provider will be unavailable or rate-limiting some of the time, with no fallback to switch to. Waiting is the only recovery.
- Scoring one resume is independent of scoring any other, so ordering does not matter and parallelism is free.
- Re-running a resume through the model costs money but is otherwise harmless — the output is regenerable, not authoritative.
- Polling a table at this rate is negligible load. At batches of 200 and a handful of workers, claim queries are indexed single-row lookups, not a scan.
- Recruiters tolerate a batch that fills in over minutes provided they can see it progressing. This is an assumption about users that we have not validated.

Constraints

- The numbers this design must hit are fixed: a two-second acknowledgement, 100 resumes in ten minutes on four workers, throughput proportional to worker count, and no accepted resume lost. Two load tests with an explanation of the results are required of us.
- One protocol on every boundary, and the Hiring / Resume Processing boundary itself, were fixed by ADR-001. A broker-driven service is expected of us eventually; that record defers it deliberately, and this one is written inside that deferral.
- Stated screening behaviour is binding: return a batch ID immediately, process each resume independently, update the ranking as results arrive, flag unreadable files without affecting the rest, retry a failing scorer and fall back to *needs manual review* rather than a zero score, and tell the recruiter when the batch is done.

Positions

18. **Synchronous REST** — the recruiter waits for the batch.
19. **One handover per batch** — Resume Processing loops over 200 resumes inside one accepted call.
20. **A claimable work table inside Resume Processing** *(chosen)* — `SELECT … FOR UPDATE SKIP LOCKED`, retry and terminal states as rows.
21. **An in-memory queue inside Resume Processing** — the handover call pushes onto a channel, a worker pool drains it, nothing is written until there is a result.
22. **RabbitMQ** — a task queue with per-message acknowledgement, retry and dead-lettering.
23. **Apache Kafka** — a partitioned, replayable log.
24. **A shared work table in Hiring's database**, polled by Resume Processing.

Argument

Why not synchronous (1). It fails on the first batch: the request outlives any sane gateway timeout, there are no partial results, there is nowhere to retry from, and the recruiter's browser becomes the thing holding the state of a five-minute job. It also makes the model's availability the API's availability.

Why per-resume rather than per-batch (2). The decision that matters most here, and the one easy to get wrong because per-batch looks simpler — one call instead of two hundred. With one handover per batch, a single corrupt file fails the unit and a retry re-processes all 200 resumes, re-paying every model call already made. There is no concurrency across replicas within a batch without building a second, in-process work distributor. Progress cannot be reported. And a batch stuck behind one pathological file occupies a worker for the duration. Per-resume work makes the failure unit the same size as the failure: one bad resume damages one result.

Why a durable table over an in-memory queue (4). This is the difference between a design that loses work and one that does not. An in-memory queue makes the handover a lie: the call is acknowledged, the process restarts, and twenty resumes the recruiter was told were accepted no longer exist anywhere. There is nothing to reconcile against, because nothing recorded that they were owed. A row committed before the acknowledgement is what makes *no accepted resume is lost* a claim we can test by killing a worker mid-batch, rather than a hope.

Why a table over a broker (5, 6). Under one protocol on every boundary, a broker is not available to us for free — it is a second piece of infrastructure and a second delivery model, adopted at the moment the boundaries themselves are still unproven. `SELECT … FOR UPDATE SKIP LOCKED` is a well-understood pattern that gives the properties this workload actually needs: independent units, at-least-once delivery, per-unit retry with backoff, a terminal state for work that will not succeed, and — the part a broker makes harder — the work item and its result in the same transaction as the domain data.

The cost is real and we state it plainly: retry, backoff, lease expiry, the reconciliation sweep and an operator's view of failed work are all code we write and test, and every one of them arrives free with RabbitMQ. There is also a ceiling. Polling costs a query per worker per interval, claim contention rises with worker count, and a table is a poor fit for fan-out or for consumers in other services. None of those binds at 200 resumes and four workers. If any of them starts to bind, the fix is a broker, and this record is the thing it supersedes.

Why Kafka specifically was never close (6). Kafka is an ordered, replayable log; our workload is unordered, independent tasks needing per-message retry and somewhere to put work that will not succeed. Consumption is per-partition and ordered, so one resume taking five minutes blocks every message behind it: head-of-line blocking, exactly the failure this record exists to prevent.

Why the table lives in Resume Processing, not Hiring (7). A shared table is two services writing one schema, which makes them one deployable wearing two names — the coupling the decomposition exists to prevent. Under the split we have, in-flight screening work is Resume Processing's own state. Hiring holds the entry and its status; Resume Processing holds the work. The two are reconciled by handover and callback, not by a join.

Where the criteria come from. The criteria snapshot travels in the handover body and is stored on the work row. Resume Processing never calls Hiring to fetch criteria. A call per resume would add a synchronous dependency to the path that exists to have none, and criteria can be edited while a batch runs — the snapshot pins one version to one resume, so every justification is defensible against the criteria actually used.

The snapshot is working state, discarded with the row once the entry is terminal; the criteria themselves belong to the job opening and stay in Hiring.

How this delivers the quality attribute. Scalability here is one falsifiable claim: *add workers and throughput rises in proportion.* Per-resume rows are what make it true — no in-process distributor to saturate, no batch-level lock, no ordering constraint, and `SKIP LOCKED` means the fourth worker steps over the rows the first three hold rather than waiting behind them. The ceiling is the provider's rate limit, not our design. The comparative load test runs the same 100-resume burst at one, two and four workers and plots completion time against worker count.

The second load test keeps the first honest: fault injection with the provider deliberately failed — zero lost rows, retries backing off, work that exhausts its budget ending in the `failed` state and nowhere else, and the batch completing once the provider returns. Throughput bought by dropping work is not throughput, and a scalability demonstration that cannot show the second graph has not shown the first.

Implications

What this buys us

- A batch degrades resume by resume instead of failing whole. Unreadable files and a failing scorer stop being special-case error handling and become ordinary states of a row.
- Throughput is a deployment parameter: more replicas, more concurrency, no code change — which is what makes the quality attribute demonstrable live rather than only in a report.
- A model outage becomes latency rather than data loss, which is what makes running without a fallback survivable at all.
- No new infrastructure. The durability, the transaction and the operator's view come from a database the service already runs, and the whole pipeline can be inspected with one SQL query.
- The work item, the profile and the score commit together, so there is no window in which a resume is marked done but its result is missing.

What it costs us

- **We are hand-building queue semantics.** Backoff, lease expiry, attempt counts, the reconciliation sweep and a replay path for failed rows are ours to write, ours to test, and ours to get wrong. A broker ships all of it. This is the central cost of the decision and it should not be minimised: the most likely serious bug in this system lives in that code.
- **No operational console.** There is no management UI showing depth, consumer count and redelivery. We need our own view of pending, claimed, failed and stalled work, or failures become invisible — and the fault-injection demonstration needs something to show.
- **At-least-once means occasionally twice.** A worker that dies after calling the model but before committing re-scores that resume and we pay again. The upsert makes the data correct; it does not make the bill correct. A duplicated callback is harmless for the same reason.
- **Two handover failure modes, not one.** The call from Hiring can fail after the row is written, and the callback can fail after the result is stored. Both are retried, and both need the receiving side to be idempotent — which is why the upsert key is stated in the decision rather than left to implementation.
- **Results are eventually consistent, and the UI must say so.** There is no moment when the answer is complete unless the screen tracks it, so the ranked list must show progress and a partial ordering, computed at read time from whatever has landed.

- **"Is this batch finished?" is a derived question** that can be answered wrongly. An entry whose handover was lost stays `pending` forever; the reconciliation sweep exists solely to bound that, and it must be written and tested or the guarantee is decorative.
- **Screening load lands on a database.** Claim queries and result writes share Resume Processing's PostgreSQL with its own reads. Contention is bounded by keeping the table narrow and indexed on *(state, next_eligible_at)*, but under load test the database is measuring itself, and that has to be accounted for when the numbers are read.
- **This design has a known end.** Fan-out to a third consumer, cross-service events, or worker counts high enough for claim contention all point at a broker. Adopting one later means rewriting the handover path in two services, not swapping a library.

Related

Related decisions

- ADR-001 — established the Hiring / Resume Processing boundary this work crosses, the AI Service the worker calls, and the single-protocol rule that ruled the broker out for now.
- **How model access is provided** — the absence of a fallback is why the retry and terminal-state policy here must be as explicit as it is.
- **Where results are stored** — the per-resume result written on each successful claim.
- **Still open:** an operator-facing view and replay path for failed work, and the reconciliation sweep's schedule and lease bounds. Both are named here as obligations, not designed.

Related requirements

- **Batch screening** in full: immediate batch ID, concurrent parse and score, streamed results, the batch-finished notification, and both failure paths — an unreadable file, and a scorer that is unavailable and must retry before falling back to *needs manual review* rather than a silent zero. Pipeline monitoring is fed by the callbacks recorded here.
- **FR-2.1** — PDF only, validated before handover. **FR-2.2** — acknowledge without waiting. **FR-2.4** — each resume independent, which the per-resume row makes structural. **FR-2.6** — ranking updated as results arrive. **FR-2.7** — retry, then manual review, excluded from the ranking rather than scored. **FR-2.12** — batch-finished notification from the computed terminal state.
- **NFR-02** — the 60-second abandon-and-retry bound defines a transient failure here. **NFR-04** — two-second acknowledgement regardless of batch size. **NFR-06, NFR-07, NFR-08** — throughput, its proportionality to worker count (**the demonstrated quality attribute**), and the burst it must sustain. **NFR-10** — no accepted resume lost, verified by fault injection.
- Required of us: the two load tests and their explanation, the demonstrated quality attribute, and the risk matrix — to which this record contributes the hand-built-retry and provider-outage risks. The broker-driven service expected of us is not delivered here.

Related artifacts

The load-test plan and the risk matrix, when written, both derive from this record.

Related principles

- *Explainability is required, not optional* — an unscored resume is marked *needs manual review* with a reason instead of receiving a zero that would silently rank it last.
- *The human still decides* — work the system cannot judge is handed back to a person, not guessed at.
- *Easily reversible* — the work table is deliberately shaped like a queue, so replacing it with one changes where rows live, not what the states mean.

Notes

This was first drafted on RabbitMQ, with per-message acknowledgement, quorum queues and a dead-letter exchange. It was rewritten the same day when we decided to run every boundary over REST and defer the broker: the queue moved inside the service and became a table. What did not change is the shape of the decision — one unit of work per resume, at-least-once, idempotent results, classified retries, a terminal state for work that will not succeed. Those are the parts that were actually argued. The broker was the implementation of them, and it is the part we expect to reinstate first.

It was also first drafted with *Reliability* as the demonstrated quality attribute, on the argument that never losing work is the design's most distinctive property. It was re-pointed at *Scalability* to match the measurement already chosen. The reliability claim survives as the fault-injection test, which the Argument now treats as the check on the scalability numbers rather than as a separate claim.

We considered a retry budget generous enough to ride out any plausible outage — hours rather than minutes — so nothing would ever reach manual review for availability reasons. Rejected because it hides the outage: a recruiter would see a batch that never finishes and no explanation. A bounded budget followed by an explicit *needs manual review* tells the truth, and the truth is recoverable — rows can be replayed once the provider is healthy.

ADR-003: PostgreSQL as system of record, MongoDB for AI-derived documents

Date: 2026-09-11 **Deciders:** Patiphon Puntusin, Thanabul Parodom, Thanwarat Korcharoenkiat, Rerngrit Jangsri

> *In the context of storing hiring records alongside data the model derives from resumes, facing two kinds of data with opposite needs — one that must never be wrong, one whose shape changes every time we improve a prompt — we decided to keep the system of record in PostgreSQL and the derived documents in MongoDB, to achieve enforced integrity where law and auditing demand it and schema freedom where the model's output evolves, accepting that one candidate's data now spans two stores and every erasure must succeed in both.*

Summary

Issue

The data splits cleanly in two, and the split is not about volume — it is about what happens when the data is wrong.

One half must never be wrong. Memberships and roles decide who may see whose candidates. A job opening's weighted criteria are what a score is defensible against; an override records that a human disagreed with the model. The retention policy determines what the company may lawfully keep, and the audit log is the evidence it did what it claims. This data is relational and stable, and needs constraints, transactions and deliberate migrations. Losing referential integrity here is not a bug, it is a compliance incident.

The other half is derived, and its shape is unsettled. A profile parsed from a resume is deeply nested and extremely sparse — three jobs or eleven, a publications list or none, skills grouped one way this month and another next. Scoring output carries per-criterion sub-scores, a must-have check and a justification, and its structure changes every time we improve the prompt. Modelling that relationally means a table per nested collection, a migration per prompt change, and a wide row of nullable columns for the fields most resumes lack.

Privacy law cuts across both. A person must be erased from *everywhere* on a schedule, and there are two expiry actions — delete outright, or anonymise while preserving the aggregate counts the dashboard is built from. Anonymisation is only clean if the identifying material and the statistics are separable in the first place.

Decision

PostgreSQL is the system of record. It holds everything whose correctness is load-bearing:

- users, sessions, memberships and roles
- job openings and their weighted criteria
- screening batches and the per-resume status of each entry
- shortlist decisions, the AI score, and any human override
- candidate identity, the date the data was collected, and the retention anchor date

- the retention policy; the append-only erasure audit log, which names a candidate only by a pseudonymous identifier and holds no personal data; and the access audit log
- the pipeline counters the dashboard reports from

MongoDB holds what the model derives. Documents, read whole and replaced whole:

- the normalised candidate profile parsed from a resume
- extracted resume text
- per-criterion scoring output with its evidence and the written justification — persisted with the score rather than regenerated on demand
- generated interview guides

Resume files live in object storage (S3-compatible; MinIO locally), not in either database. PostgreSQL holds the object key.

The rule for future data, recorded so the boundary does not rot:

> *If getting it wrong is a correctness or compliance problem, it goes in PostgreSQL. If the model produced it and its shape changes when the model or the prompt changes, it goes in MongoDB. Nothing in PostgreSQL may depend on a MongoDB document for its correctness.*

Linking and erasure. Every MongoDB document carries `workspace_id` and `candidate_id` and is indexed on both, so retention deletes a person's derived data directly rather than by traversing relationships. PostgreSQL is authoritative for the existence of a candidate; MongoDB is authoritative for nothing.

Ownership follows the service boundary set by ADR-001: each service owns its own schema and no service reads another's tables. Two services needing the same fact exchange it in a call, not a join.

Status

Accepted

Group

Data

Details

Assumptions

- Volume is modest — thousands of candidates per deployment, not millions. Neither store is chosen for scale; both are chosen for fit.
- Derived documents are read whole and written whole, so document semantics lose us nothing.
- MongoDB documents are regenerable from the stored resume file, right up until retention deletes that file. After erasure nothing is regenerable, which is the intended behaviour.
- The team can run both stores locally and in the cluster — a real load on a development machine alongside six services.
- Both are open source and free to run, so licensing does not enter the decision.

Constraints

- At least two database types, one relational and one NoSQL, are required of us.
- Erasure must be complete and provable, so every store holding personal data must be reachable by the retention process and must appear in the audit trail.
- Per-service data ownership was fixed by ADR-001; this decision operates inside it.

Positions

25. **PostgreSQL only, with `jsonb` columns** for profiles and scoring output.
26. **MongoDB only** — one store, no migrations, everything a document.
27. **PostgreSQL + Redis** as the second database type.
28. **PostgreSQL + Elasticsearch** — profiles indexed for full-text search.
29. **PostgreSQL + MongoDB** *(chosen)*.
30. **One shared database for all services**, instead of per-service ownership.

Argument

PostgreSQL-only with `jsonb` (1) deserved the most argument, because it is genuinely strong: schema freedom without a second store, real transactions across the record and the derived data, one backup, one set of credentials, one thing to learn.

It was rejected primarily because of the *anonymise* action. When retention expires and the policy says anonymise rather than delete, the company keeps the statistics and destroys the person. If identifying material lives in `jsonb` on the same rows as the counts, anonymisation becomes a surgical rewrite of every row — one missed column away from a compliance failure. With derived documents in a separate store it is *drop this candidate's documents, keep the rows*: coarse, verifiable, easy to prove.

Second, blast radius. Scoring output changes shape most often and is rewritten most carelessly; keeping it out of the database that holds the retention policy and the audit log means an experiment with the parser cannot lock or bloat the table that proves we complied with the law.

Third, the two-database requirement — listed third deliberately. Without it, position 1 would have been defensible and we would have had a harder argument.

MongoDB-only (2) fails in the opposite direction. Memberships, retention policy and the audit log are exactly the data that must not drift, and we would be hand-rolling referential integrity in the one area where being wrong is expensive. Multi-document transactions exist, but choosing a store whose defaults work against the guarantee you most need is a poor trade.

PostgreSQL + Redis (3): Redis is a cache, not a store. Using it as the second database type means either treating a cache as durable — the classic way to lose data quietly — or nominating a "second database" holding nothing anyone would miss.

PostgreSQL + Elasticsearch (4) solves a deferred problem. Full-text ranking across the candidate archive belongs to re-matching, which is parked; nothing in the current use cases searches resume text. Adopting a search cluster now buys a capability nothing uses and costs memory on a cluster already carrying six services and two databases.

One shared database (6) would undo ADR-001 from below: services sharing tables are not independently deployable, and the first cross-service join silently makes two services one.

Implications

What this buys us

- Constraints do work that application code cannot afford to do: a candidate cannot belong to no batch, a decision cannot reference a job opening that does not exist.
- Anonymisation becomes coarse and auditable rather than a field-by-field rewrite — *drop the documents, keep the counts*.
- Changing the parser or a prompt changes a document shape, not a schema: no migration, no coordinated release.
- The audit log lives in an append-only relational table where its integrity is enforced, which is what makes it usable as evidence.

What it costs us

- **There is no transaction across the two stores.** A candidate row can exist with no profile document and — the case that matters — a profile document can survive a successful delete. Erasure must therefore be a sequenced operation with a verification pass and a *deletion pending* state, completing within the 30 days allowed. Deleting the object-storage file is a third step with the same problem.
- **Two stores to back up, monitor, secure and hold credentials for**, and two places a breach can happen. The privacy surface is larger than with one store — the honest cost of the anonymisation benefit above.
- **The dashboard cannot join across the split.** Metrics must be maintained in PostgreSQL from services as work completes, not computed on demand. More code, and a lost update is a permanently wrong number until something reconciles it.
- **The split rule has to be enforced by people.** The first time someone puts a decision reason in MongoDB "because it was easier", the boundary starts rotting and the anonymisation argument stops being true. This is a review responsibility, not an assumption.
- **Two data models for four students to learn**, including the parts that bite: migrations and connection pooling on one side, index behaviour and document-size limits on the other.
- **Referential integrity stops at the service boundary anyway** — a foreign key cannot span services, so some invariants are enforced by convention and by calls whichever store holds them.

Related

Related decisions

- ADR-001 — per-service data ownership, which this record refines into a concrete store-by-store split.
- ADR-002 — writes exactly one derived document per successfully completed unit of work and one status transition per entry.
- **How model access is provided** — the model version recorded with each score is stored beside it, which is what makes an old justification reconstructable.

Related requirements

- **Sign-in and access management** (users, sessions, memberships, roles), **job creation** (openings and criteria), **screening** (batches, profiles, scores, justifications, overrides, decisions), **interview questions** (guides), **pipeline monitoring** (counters) and **retention** (policy, audit log, both expiry actions) all map onto the split above.
- **FR-0.3, FR-0.4** — roles as a foreign key, not a convention. **FR-1.7** — the opening, its criteria and the original description. **FR-2.5, FR-2.8, FR-2.9** — score, must-have check, override and shortlist decision, all in PostgreSQL so the ranked list never depends on a document read. **FR-2.11** — collection date. **FR-3.1** — guides retained. **FR-5.1** — retention policy. **FR-5.3 to FR-5.5** — erasure across both stores and object storage, anonymisation as *drop the documents, keep the counts*, and an audit log that names no one. **FR-5.8** — deletion pending.
- **NFR-03** — ranked list served from PostgreSQL columns within two seconds. **NFR-11** — encryption at rest across both stores *and* object storage: three places to configure, not one. **NFR-12** — access audit log, append-only. **NFR-13** — the erasure sequence completes within 30 days. **NFR-15** — per-criterion evidence persisted with the score.
- Required of us: two database types, with a stated reason for what lives in each.

Related artifacts

The Service–Operations–Collaborators table will name the store each service owns.

Related principles

- *Explainability is required, not optional* — the justification is stored with the score, not regenerated on demand, so what a recruiter saw is what can be reviewed later.
- *Easily reversible* — collapsing MongoDB into `jsonb` later is a migration; splitting a single store after retention logic has been written around it is not.

Notes

Recorded plainly, because a future reader will suspect it anyway: the two-database requirement is what forced the question. What we did with it was not arbitrary — the anonymisation argument stands on its own and would have justified the split regardless — but it is fair to say we would probably have shipped `jsonb` first and reached this split only when anonymisation was implemented.

Redis is likely to appear later as a cache for model responses. When it does, it is a cache and must be treated as one: nothing may be true only in Redis. That is a note, not a decision, and needs its own record if it happens.

ADR-004: All model access through one AI Service, on a managed API that does not train on our data

Date: 2026-09-11 **Deciders:** Patiphon Puntusin, Thanabul Parodom, Thanwarat Korcharoenkiat, Rerngrit Jangsri

> *In the context of every AI behaviour in the product, facing resumes that are personal data and a team with no GPU capacity, we decided to route all model calls through a single internal AI Service speaking to a managed API whose terms exclude training on our inputs, to achieve a disclosure we can lawfully justify and one place to control cost, prompts and failure, accepting a dependency we cannot repair ourselves and periods during which screening produces no scores at all.*

Summary

Issue

Three behaviours need a language model: turning a paragraph of plain Thai or English into weighted criteria, scoring a profile against those criteria with a justification, and generating interview questions from the intersection of a resume and a job opening. The model is not a feature of HireAssist; it is the capability the product is sold on.

What we send it is personal data. A resume carries a name, contact details and employment history. Sending it to a third party is a disclosure needing a lawful basis and terms bounding what the recipient may do with it. One clause decides everything: if the provider may train on submitted data, every resume becomes part of a model weight we cannot reach into and delete, and our promise that a candidate's data is erased on a schedule would be false from the first batch. A provider that trains on our inputs is not a cheaper option; it is an incompatible one.

We have no GPU and no budget for one. Nobody has hardware that runs a competitive model, and our customers are price-sensitive, so per-token cost is architectural rather than operational: it scales linearly with applicant volume, which is the very thing our customers have too much of.

Resumes arrive in Thai and English, often mixed in one document. A model that handles Thai poorly produces exactly the failure the product exists to prevent — a good candidate rejected for their vocabulary.

The model is the least reliable dependency in the system, yet we have committed to never losing an accepted resume whatever it does, and to a bound on how long one scoring attempt may wait. Rate limits, latency spikes and outages are certain. The provider's rate limit, not our worker count, may also turn out to be the ceiling on the throughput we must demonstrate.

Decision

1. Every model call goes through the AI Service. No other service holds a model credential or a prompt. The AI Service exposes REST operations in domain terms, not model terms:

- `POST /criteria-extractions` — *(free text, language)* → proposed title, weighted criteria (UC-1)
- `POST /profile-scores` — *(candidate profile, criteria)* → score, must-have check, justification (UC-2)
- `POST /interview-guides` — *(candidate profile, criteria, emphasis)* → tagged questions (UC-3)

Callers ask for a screening score, never for a completion. The provider, the model name, the prompt and the token budget do not appear in any other service's contract. This is NFR-17 made structural: the scoring model or prompt changes without modifying or redeploying anything else.

2. The provider is a managed API whose terms exclude training on our inputs and outputs, with a stated retention window and a data-processing agreement we can put in front of a customer. The initial choice is the **Anthropic Claude API**, which does not train on API inputs by default; the OpenAI and Google Gemini paid API tiers carry comparable commitments and remain substitutes. The provider is configuration, not code.

3. There is no fallback model. When the provider is unavailable the system does not switch to a weaker one. Work waits in the caller's work table, retries with backoff, and — if the retry budget is exhausted — the affected resumes are marked *needs manual review*, exactly as ADR-002 specifies and as UC-2's alternate flow 4a already promised.

4. Cost and exposure are controlled inside the AI Service:

- **Minimise what is sent.** The prompt receives the parsed profile fields the operation needs and the criteria — never the raw file, never fields irrelevant to the judgement, and never the attributes NFR-14 excludes from scoring. Gender, age, marital status, religion, nationality and photograph are stripped from the profile inside the AI Service, before any prompt is built, so that no caller can forget to.
- **Cache by content.** Results are keyed on the profile version and the criteria version, so re-running a batch or re-opening a guide does not re-pay for an identical call.
- **Budget per deployment and per batch.** A runaway batch hits a ceiling and reports it rather than producing an invoice.
- **Meter every call** — model, version, token counts, latency, outcome. The metering log contains no personal data.

5. The model version is recorded with every score and justification. Providers update models underneath a stable name; without the version pinned and stored, a score from March cannot be explained in September. NFR-15 requires the per-criterion evidence to be persisted with the score; the version is what makes that evidence reproducible rather than merely retained.

6. The provider is documented as a data processor. The customer's privacy notice and the consent text must disclose that resumes are processed by a named overseas processor. This is a product obligation created by this decision, not a footnote to it.

Status

Accepted

Group

AI/LLM · Security · Cost

Details

Assumptions

- The provider's terms genuinely exclude training on API inputs, and the retention window is short and documented. **This is the assumption the whole decision rests on and it must be verified against the**

provider's current data-processing terms before any real resume is processed — not taken from this document, which will age.

- A frontier hosted model reads Thai résumés well enough to score them fairly. Not yet validated; it needs a small evaluation set of real-shaped (synthetic) Thai resumes before we rely on it.
- Per-token cost stays within what the pricing in the proposal can absorb at 20–200 resumes per opening.
- Provider availability is high but not total, and outages are minutes to hours rather than days. If that proves wrong, position 3 below is reopened.
- Team has no GPU, and the term project has no hardware budget.

Constraints

- Personal data may be disclosed to a processor only under terms bounding its use, and must remain erasable on request or on expiry.
- Customers are price-sensitive, so per-candidate cost is bounded by what an SME will pay.
- ADR-001 established the AI Service as a separate service, reached over REST like every other boundary and not exposed through the gateway.
- ADR-002 fixed how failures of this dependency are handled.

Positions

31. **Self-hosted open-weight model** (Llama, Qwen, Typhoon and similar) on Ollama or vLLM — nothing ever leaves our infrastructure.
32. **Managed API with a self-hosted fallback** — the hybrid: use the API normally, fail over to a local model during an outage.
33. **Two managed providers with automatic failover** between them.
34. **A cheaper provider whose default terms permit training, with training opted out** in the account settings.
35. **Managed API only, one provider, mediated by our own AI Service** (chosen).
36. **No AI Service — each service calls the provider SDK directly** where it needs a model.

Argument

Self-hosting (1) is the strongest privacy answer and we still rejected it. Nothing leaving our infrastructure would remove the disclosure question entirely and make PDPA compliance substantially easier to argue. It fails on capability rather than principle: there is no GPU and no money for one, and the models small enough to run without one are materially weaker on Thai — which converts a privacy win into a fairness loss in the one place the product must not fail. It also moves the availability problem to four students who would then be the on-call team for a model server, which is not obviously more reliable than a provider with an SLA. Worth re-examining if the project ever has hardware; the AI Service boundary is what would make that re-examination cheap.

The hybrid (2) was reconsidered because of the promise never to lose accepted work, and rejected anyway. A fallback model makes a candidate's score depend on which provider happened to be healthy when their resume was processed. Two candidates in the same batch could be ranked against each other on scores produced by different models — indefensible in a system whose entire proposition is a comparable, justified score, and unanswerable when a recruiter asks why one was rated lower. It also doubles the prompt-tuning and evaluation surface for a four-person team. The promise we can honestly make is

narrower: *every accepted resume reaches a terminal state* — scored, or flagged for manual review — not *the AI is always available*. The work table holds the work; the recruiter is told; nothing is lost or silently guessed.

Multi-provider failover (3) has the same comparability problem in a milder form, plus two sets of terms to verify and keep verified. Deferred rather than dismissed: because every call is already behind the AI Service's domain operations, adding a second provider later is a change in one service. If outages turn out to be frequent, this is the first thing to revisit.

Opt-out terms (4) were rejected on the difference between a setting and a contract. An opt-out can be reset by an account change, a plan migration, or someone else on the team; a contractual term cannot. Where a compliance promise is being made to a customer, the guarantee has to be the kind you can show someone, not the kind you have to remember to keep switched on.

Direct SDK calls from each service (6) would put the credential in three codebases, the prompts in three places to drift apart, and no single point at which spending can be capped or a provider swapped. Everything this record decides in items 2–5 would have to be implemented three times and would be inconsistent within a month.

What decided it, in order: the training clause is non-negotiable because the erasure promise depends on it; hardware we do not have rules out self-hosting; Thai-language quality rules out the small models we could otherwise run; and comparability of scores rules out mixing models. What remains is a single managed provider — so the design work goes into making that dependency safe to have: minimise what is sent, cache what repeats, cap what it costs, record what produced each answer, and let the caller's retry policy absorb its failures.

Implications

What this buys us

- One credential, one prompt set, one cost centre, one place a provider swap happens.
- A lawful basis we can actually put in a privacy notice: named processor, no training, documented retention.
- The erasure promise stays true, because nothing we send is absorbed into a model.
- Every score carries the model version that produced it, so a justification can be reconstructed months later — the auditability requirement, satisfied structurally.
- Any future re-matching over the candidate archive reuses `ScoreProfile` unchanged and inherits the same budget and caching controls that keep it from multiplying cost.

What it costs us

- **During a provider outage, screening produces nothing.** Not slower results — no results. The work table holds the work and the recruiter waits; if the outage outlives the retry budget, those resumes land in *needs manual review* and a human has to do what the product was bought to avoid. This is the direct, accepted consequence of having no fallback, and it belongs in the risk matrix.
- **Cost scales linearly with applicants.** A customer with 2,000 applicants costs roughly ten times one with 200, while both may pay a similar subscription. Either pricing is per-volume or margins invert on exactly the customers the proposal says we serve best.
- **Personal data crosses a border.** Some customers will refuse on that basis alone, and the consent text must state it. This narrows the addressable market in a way a self-hosted option would not.

- **We cannot fix a quality regression.** When the provider updates a model, scores can shift under us with no code change on our side. Pinning the version limits it, but pinned versions are eventually retired, and a migration means re-validating every prompt.
- **Budget ceilings create a new failure mode**: a batch that stops mid-way because a deployment exhausted its allowance. That state has to be visible and resumable, or it looks identical to a bug.
- **The AI Service is a bottleneck for everything.** Interactive criteria extraction waits behind batch scoring unless the service separates them: the two need different rate-limit budgets, which is more machinery in the one service already holding the credential. And the provider's rate limit, not our worker count, may become the ceiling on the scalability demonstration — a number we do not control, and the reason a deterministic pre-filter remains a candidate decision.
- **Caching stores derived personal data.** A cache keyed on profile content is another place a candidate's data lives, so it must respect retention and be purged with everything else, or the cost optimisation quietly reopens the compliance hole.

Related

Related decisions

- ADR-001 — created the AI Service, put it behind REST like every other boundary, and made it unreachable from the gateway; that boundary is what this record fills in.
- ADR-002 — carries the failure handling this decision relies on. The two records are a pair: no fallback model is only acceptable because no work is lost.
- ADR-003 — scoring output and the recorded model version are stored as documents in MongoDB.

Related requirements

- **Criteria extraction**, **scoring with a justification** and its retry path when the model cannot answer, **interview guide generation**, and the **erasure promise** the no-training term protects. Any future re-matching depends on `ScoreProfile` being reusable.
- **FR-1.2** — criteria derived from free text. **FR-2.5** — score, must-have check and justification for every resume. **FR-2.7** — retry, then manual review. **FR-3.1 to FR-3.4** — guide generation and regeneration with a stated emphasis. **FR-5.3** — erasure reaches the cache too.
- **NFR-02, NFR-05** — scoring and guide latency bounds, largely set by the provider rather than our code. **NFR-07** — the provider's rate limit as a possible ceiling on the demonstrated attribute. **NFR-10** — no accepted resume lost, whatever the provider does. **NFR-13** — erasure within 30 days, including cached derived data. **NFR-14** — protected attributes stripped inside this service before any prompt is built. **NFR-15** — per-criterion evidence, reproducible because the model version is stored with it. **NFR-16** — Thai and English. **NFR-17** — model and prompt change within this one service.
- Required of us: the demonstrated quality attribute, for which this record names a ceiling we do not control; and the risk matrix, to which it contributes provider outage, rate-limit ceiling, cost escalation, cross-border transfer and model drift.

Related artifacts

The privacy notice and consent text, which do not exist yet, are a deliverable this decision creates.

Related principles

- *The human still decides* — reinforced here: when the model cannot answer, the work goes to a person rather than to a guess.
- *Explainability is required, not optional* — a score without a reproducible justification is not acceptable, which is why the model version is stored with it.
- *Easily reversible* — the provider is behind a domain-shaped interface, not a model-shaped one, precisely because it is the part of this decision most likely to be wrong. The three operations above are the contract; the wire protocol carrying them is not.

Notes

The tension here was raised explicitly and is worth preserving. We promise that no accepted resume is ever lost, then choose a design with a single point of AI failure and no fallback. That is deliberate: the promise is about the *system*, not the model. Work submitted is never lost, no failure of the model takes down sign-in, the dashboard, or an in-flight batch, and every resume the model could not judge ends in an honest state a human can act on. A fallback model would have made a different claim — that a score is always produced — at the price of scores that are not comparable to each other. We would rather be slow and consistent than fast and incoherent.

Self-hosting was the position the team was most reluctant to reject; it is recorded as the first thing to reconsider if hardware appears, and the AI Service boundary exists partly to keep that door open.

ADR-005: Each service chooses its own language and framework, within shared contracts

Date: 2026-09-11 **Deciders:** Patiphon Puntusin, Thanabul Parodom, Thanwarat Korcharoenkiat, Rerngrit Jangsri

> *In the context of five services owned by four developers whose strengths and ecosystems differ, facing a choice between one backend language for everything and a language chosen per service, we propose that **each service picks its own language and framework within a fixed set of shared contracts and operational rules**, to achieve ecosystem fit where it matters and to let each member work and learn where they are productive, accepting duplicated cross-cutting code, a heavier build, and a worse bus factor than a single-language system would have. Proposed, not yet accepted — the languages themselves are deliberately left open.*

Summary

Issue

ADR-001 recorded, in its Assumptions, that "Go is the backend language for all services". That was never decided. It was written as an assumption, carried no alternatives and no argument, and it is not what the team intends to do — members differ in what they know and in what they want to learn, and two of the five services sit in an ecosystem where Go is the weaker choice.

It also sat awkwardly against ADR-001's own reasoning. One of the three arguments for splitting the system at all is that four developers with different strengths can own services and work in parallel. If every service is written in the same language regardless of fit, that argument is decorative: the boundaries exist but nobody works differently because of them.

The forces:

- **Ecosystem fit is not uniform across the services.** Parsing PDFs and calling a language model are Python's home ground — `pypdf`/`PyMuPDF` for extraction, mature provider SDKs, tokenisation and prompt tooling. Writing those two in Go means reimplementing or wrapping what already exists. The other three services are ordinary transactional web services that any mainstream stack does well.
- **The team is four part-time developers with different strengths**, under a deadline, who also need to *learn* from this project — it is a course, not only a delivery.
- **Contracts already cross every boundary** — every service call is REST over HTTP/JSON, at the gateway and between services alike. They must be explicit whether or not the languages differ.
- **Whatever we choose, four people must still operate it**, correlate its logs, and cover for each other in demo week.

Decision

Each service chooses its own language and framework. Its owner decides.

No language is fixed by this record. What is settled is that a single mandated language is not assumed, and what conditions a per-service choice must meet. The choices themselves wait on service ownership, which is not yet assigned.

That freedom is bounded by rules that apply to every service regardless of language:

37. **Contracts are written once and generated from, never hand-written twice.** Every boundary — the gateway's and every service-to-service call — is defined by an OpenAPI document, and clients and servers are generated from it in each language. A contract written by hand in two languages drifts. With no schema-carrying wire format underneath, the OpenAPI document is the only thing preventing that, so it is authoritative rather than descriptive: it is written before the endpoint, and a contract test on each side proves both still match it.
38. **Schema definitions live in one place**, versioned with the system, not copied into each service. Exactly where is part of the still-open repository-structure decision.
39. **One observability standard.** Structured JSON logs with a shared field set, and OpenTelemetry traces propagating the same correlation id, carried as an HTTP header on every call. Every mainstream language has an OTel SDK; without this, a resume's journey cannot be followed across five services in three languages.
40. **Every service runs the same way.** A Dockerfile, a health endpoint and a readiness endpoint, startup configuration from the environment, and a Kubernetes manifest. The platform must not need to know what is inside the container.
41. **Every service has a primary and a secondary owner**, and they are named in the repository. The secondary must be able to build, run and deploy the service. Polyglot makes this rule mandatory rather than advisable: a teammate cannot improvise in a language they have never used, on the morning of a demo.
42. **A new language needs a reason recorded here.** Ecosystem fit or an owner's genuine strength qualifies; curiosity about a fourth stack in week nine does not.

Allocation is deliberately not decided here. The table below records where the ecosystem argument is strong and where it is not, as input to the choice — not as the choice:

Service	Ecosystem pull	Note
AI Service	Strong — toward Python	Provider SDKs, tokenisation and prompt tooling are first-class there; this service changes most often (NFR-17) and benefits most from the shortest edit-run loop
Resume Processing	Strong — toward Python	PDF text extraction and Thai/English text handling have the strongest libraries there (FR-2.3, NFR-01)
Hiring Service	Weak	Ordinary transactional service with the largest domain surface; the owner's fluency will matter more than the ecosystem
Identity & Workspace Service	Weak, with a caveat	Security-sensitive and small; whatever is chosen should have mature, well-trodden auth libraries

Compliance & Insights	None	Scheduled work and event consumption; no unusual library needs
API Gateway	None	May be an off-the-shelf gateway rather than code we write

Status

Proposed.

The principle and the guardrails are ready; the choices are not. Before this can move to *Accepted*:

43. Each service needs a **primary and secondary owner** (rule 5) — the owner is who makes the choice, so nothing can be chosen until ownership exists.
44. The team confirms it accepts the cost of more than one language, and agrees an upper bound on how many.
45. Each proposed language is checked against the gate in *Assumptions* — a maintained OpenAPI generator, an OpenTelemetry SDK, and clients for whichever of PostgreSQL and MongoDB that service needs.

Until then, ADR-001's assumption that a single language is used is withdrawn, and no replacement is in force.

Group

Implementation · Team

Details

Assumptions

- Each of the five services has, or will have, a primary owner among the four members.
- The team is willing to pay a per-language cost for cross-cutting concerns rather than share one internal library.
- Every language chosen has a maintained OpenAPI code generator, an OpenTelemetry SDK, and a client for PostgreSQL and MongoDB as its service requires. This is true of every mainstream candidate, but it is a gate on any unusual one.
- No more than three languages across the system. This is not a rule, but beyond three the costs below stop being manageable for four people.

Constraints

- Four part-time developers and a fixed deadline. Time spent fighting an unfamiliar toolchain is time not spent on the deliverable.
- Everything must run on a laptop for the demo, alongside PostgreSQL and MongoDB.
- Contracts cross language boundaries regardless of this decision, because the services call each other whatever they are written in.

Positions

46. **One language for all services — Go.** What ADR-001 assumed. Cross-cutting concerns are written once and shared as an internal module; any member can read and fix any service; one toolchain, one CI configuration, one set of base images.
47. **One language for all services — Python.** Same benefits, and it suits the two AI-facing services; weaker for the transactional services and for long-running processes.
48. **Language per service, chosen by its owner, within shared contracts.** *(Selected.)*
49. **Two languages by tier** — Python for the AI-facing pair, one fixed language for the other three, mandated rather than chosen.
50. **Per-service choice with no guardrails.** The same freedom without the contract, telemetry and packaging rules.

Argument

Against a single language (1 and 2): whichever is chosen is the wrong one for part of the system. Go for the AI Service means wrapping provider SDKs and PDF libraries that already exist in Python; Python for Hiring and Identity means giving up static typing on the services with the largest domain surface. It also hollows out one of ADR-001's three reasons for decomposing at all: if nobody works differently because of the boundaries, the team-structure argument was never real.

Against mandating two languages by tier (4): it captures most of the ecosystem benefit and is genuinely close to what we chose. It was rejected because the mandate buys little that the guardrails do not already buy, and it overrides the owner on the three services where ecosystem fit is not the deciding factor — exactly where an owner's fluency is what determines whether the service ships. Position 3 reaches the same allocation by ecosystem where ecosystem matters, and by the person doing the work where it does not.

Against unbounded polyglot (5): freedom without generated contracts is how two services end up disagreeing about a field name in week ten, and without a shared telemetry standard a failed resume cannot be traced across three runtimes. The guardrails are what make the freedom affordable; they are not bureaucracy attached to it.

For per-service choice (3): the ecosystem argument is decisive on two services and irrelevant on three, which is precisely the shape a per-service decision fits. The contracts that would carry the cost of polyglot already exist for other reasons — one REST contract per boundary, gateway and internal alike — so the marginal cost is generating clients rather than inventing an integration style. It is thinner cover than it looks: a single wire format with no schema in it means the OpenAPI document is doing all the work, and rule 1 is load-bearing rather than tidy. And it makes ADR-001's team-structure argument true: each member works and learns where they are strongest, which for a course project is part of the point rather than a concession.

Implications

What this costs us, accepted with open eyes:

- **Cross-cutting code is written once per language, not once.** Authentication middleware, structured logging, tracing setup, error mapping, retry and backoff, and workspace-identity propagation all get a second implementation. Some of it will diverge, and the divergence will be discovered in integration rather than in review.

- **The bus factor gets worse, and the mitigation is mandatory rather than advisable.** A teammate can read unfamiliar code in a language they know; they cannot debug an unfamiliar framework in a language they have never written, under demo pressure. Rule 5 exists because of this decision.
- **Build, CI and images multiply.** A pipeline, a dependency manager and a base image per language, each with its own security updates.
- **Role enforcement is now implemented more than once.** ADR-001 warns that enforcement cannot live only at the gateway; with several languages the check exists in several codebases and must be verified in each. This is the highest-risk consequence of this decision: a privilege mistake in one implementation is one the other implementations would have prevented.
- **Local development is heavier.** Contributors need more than one toolchain installed, or a container-only workflow — which slows the edit-run loop on the services they are not carrying.
- **A shared fix is no longer one change.** A bug in correlation-id propagation is fixed in every language that has it.

What it buys us:

- Each service uses the ecosystem that actually fits its work, most clearly on the two services where the model and the file parsing live.
- Each member works and learns where they are productive, and four people can work in parallel without contending on one toolchain.
- The contract discipline the guardrails impose would improve the system even if every service shared a language.

Related

Related decisions

- ADR-001 — its Assumptions asserted a single backend language; that assumption is **withdrawn and replaced by this record**. Its argument for decomposition from team structure depends on this decision to be more than decorative, and its choice of one protocol on every boundary is what keeps the generated-contract cost to a single toolchain per language.
- ADR-002 — the screening handover and its result callback now cross language boundaries, so both contracts and the correlation id must be defined outside any one service.
- ADR-003 — each service already owns its own schema and no service reads another's tables, so differing database clients per language cost nothing extra.
- ADR-004 — one service holding the model credential is what allows the provider SDK to exist in exactly one language.
- **Still open, and directly affected:** the repository structure and where the shared OpenAPI definitions live; and the assignment of a primary and secondary owner to each service, which this decision makes a prerequisite rather than a nicety.

Related requirements

- **NFR-17** — the scoring model or prompt changes without touching any other service; a Python AI Service shortens that loop.
- **NFR-01, FR-2.3** — resume parsing, the other ecosystem-driven choice.

- **NFR-10** — no accepted resume lost, which depends on retry and acknowledgement behaviour now implemented in more than one language.
- **FR-0.3, FR-0.4, NFR-12** — role enforcement and the access audit log, plus workspace isolation while it stood, now implemented in several codebases.
- A spread of communication styles is expected of us eventually; whatever protocols arrive later, this decision requires their contracts to be generated rather than hand-written.

Related artifacts

- ADR-001, whose single-language assumption this record withdraws
- The repository structure and CI configuration, once implementation starts

Related principles

- *Contracts before convenience* — a boundary is only as good as the definition that crosses it.
- *Ecosystem fit where it matters, the owner's fluency where it does not.*
- *No freedom without a corresponding rule* — the guardrails are what make per-service choice affordable rather than reckless.

Notes

This record exists because a reviewer asked where the language had been decided, and it had not been: it sat in ADR-001's Assumptions as a settled fact with no alternatives and no argument. Worth noting how quietly that happened — Assumptions is exactly where a decision can hide.

This record is deliberately left *Proposed*. The team's position is that services may differ in language, which is enough to withdraw ADR-001's assumption; it is not enough to choose. Recording the reasoning now and the choice later is the point of having a status field — the alternative would be either an Accepted record asserting languages nobody has agreed to, or no record at all and the same assumption quietly reappearing somewhere else.

The strongest objection to this record is that three languages across five services is a lot for four part-time developers, and that a single language would ship faster. That is probably true of the first month. It was not chosen because the ecosystem cost falls on the two services that are hardest to get right and change most often, and because a single language would have quietly removed one of the three reasons ADR-001 gives for decomposing at all.

ADR-006: One deployment per customer company

Date: 2026-09-11 **Deciders:** Patiphon Puntusin, Thanabul Parodom, Thanwarat Korcharoenkiat, Rerngrit Jangsri

> *In the context of selling HireAssist to companies that each hold their own candidates' personal data, facing a choice between one shared installation serving every customer and one installation per customer, we decided to **deploy a separate instance of the whole system for each customer company** to achieve isolation by construction rather than by code, accepting that operating, upgrading and monitoring the product is now multiplied by the number of customers.*

Summary

Issue

Every use case reads or writes candidate personal data belonging to one company. Until now the documents assumed a shared installation in which a **workspace** identified the company, and in which isolation between companies was a property the code had to maintain: a workspace id on the session (FR-0.1), on every row and document, and on every internal call.

That assumption was never decided. It arrived with the multi-tenant SaaS framing in *Target Customers* and propagated into the requirements and four ADRs without anyone weighing it.

The forces:

- **A leak between customers is the worst failure this product can have.** One company reading another's candidates is a PDPA breach involving a third party's personal data, reported by our customer, caused by us. There is no partial version of this failure.
- **Enforcement would be spread wide.** ADR-001 notes that tenant identity must travel on every internal call and be enforced by each service, not only at the gateway. ADR-005 makes it worse: the check would exist in several codebases, in more than one language.
- **We are four part-time developers**, none with production multi-tenancy experience, on a deadline.
- **Customer count is small and slow-growing.** The proposal targets SMEs bought one at a time, not self-service signups arriving overnight.
- **Customers differ in what they may accept.** Some will require that their candidate data never shares a database with another company's.

Decision

Each customer company gets its own deployment of HireAssist — its own services, its own PostgreSQL and MongoDB, and its own object storage. A deployment serves exactly one company, and there is no concept of a tenant inside it.

Consequently:

- **The workspace concept is removed.** A session identifies a user and a role, not a company (FR-0.1). There is no workspace id on a row, a document or an internal call.

- **Isolation between companies is a property of deployment, not of code.** No query filter, no row-level policy, and no per-call tenant check stands between one company's data and another's, because the other company's data is not in the database.
- **The workspace-scoping requirement is withdrawn.** It required data access to be scoped to the workspace in the session. There is nothing left for it to scope.
- **Configuration that was per workspace is now per deployment** — the retention policy (FR-5.1) most of all.
- **Provisioning a customer means provisioning an instance**, and creating its first Admin account (UC-0). That is a development-team operation, not a feature.

Multi-tenancy is not ruled out for the future; it is out of scope for this project.

Status

Accepted

Group

Deployment · Security

Details

Assumptions

- Customers are onboarded individually, by us, at a rate measured in weeks rather than minutes.
- The number of customers during this project is one — the demo — and small afterwards.
- A deployment is cheap enough to run per customer. The system is five services, two databases and object storage; at SME load none of them needs to be large.
- Nobody needs to query across customers. No cross-customer reporting, benchmarking or shared talent pool is in scope. *(This is the assumption most likely to be wrong commercially — see Implications.)*

Constraints

- Four part-time developers with no production multi-tenancy experience and a fixed deadline.
- PDPA liability for a cross-customer leak falls on us as processor and on our customer as controller.
- Everything must still run on one laptop for the demo — a single deployment, which this decision makes the normal case rather than a special one.

Positions

51. **Shared installation, workspace id on every row and every call.** What the documents assumed. One deployment to operate, one upgrade to run, and cross-customer reporting stays possible. Isolation is a property the code must maintain everywhere, forever.
52. **Shared installation, database per customer.** One set of running services, but each customer's data in its own database, chosen per request from the session. Removes the query-filter class of bug but keeps a routing decision on every call.
53. **One deployment per customer.** *(Selected.)*
54. **One deployment per customer now, designed so a shared installation is possible later** — carry a tenant id through the schemas and contracts but always populate it with one value.

Argument

Against the shared installation (1): the cost of getting it wrong is unbounded and the cost of getting it right is paid in every service, every query and every call, forever. A workspace filter omitted from one query in one service leaks candidates between companies, and nothing about the system makes that omission loud — the query returns results, the screen renders, and only the data is wrong. ADR-001 already flags that gateway-only enforcement is insufficient; ADR-005 puts the same check in several languages. For a team of four with no prior multi-tenancy experience, that is the highest-risk property in the whole design, and it is risk we are not obliged to take.

Against database-per-customer in a shared installation (2): it removes the worst failure mode and keeps one set of services to operate, which is genuinely attractive. It was rejected because the routing decision is still made on every request from session data, so a bug in resolving the connection has the same consequence as a missing filter; and because the operational benefit — one upgrade — matters at a customer count we do not have.

Against carrying a dormant tenant id (4): it buys future optionality at the cost of present clarity. A field that always holds one value is not exercised, so it is not correct: the first time a second value appears, every code path that ignored it is a bug. Recording this decision is a cheaper way to keep the option than writing unused plumbing.

For one deployment per customer (3): isolation stops being something the code must achieve and becomes something the deployment already is. The failure mode that worried us most cannot occur, because the data is not present to leak. It also simplifies what remains: no tenant id in schemas, contracts; no per-call resolution; one less thing every service must get right in whichever language it is written in. At our customer count, the operational cost of separate instances is smaller than the engineering cost of the guarantee it replaces.

Implications

What this buys us

- The cross-customer leak cannot happen. This is a guarantee by construction, which is the only kind worth making about personal data.
- Schemas and contracts lose a field, and every service loses a check.
- A customer who demands their data never share a database with another company's is satisfied by the architecture, not by an explanation.
- Per-customer configuration — retention policy above all — is configuration, not data.
- Restoring one customer's backup, or deleting one customer entirely, is an operation on one instance rather than a surgical query across shared tables.

What it costs us

- **Operating the product now multiplies by customer count.** Deployments, upgrades, migrations, certificates, backups, monitoring and on-call all scale linearly. Ten customers is ten upgrades. This is the cost this decision buys the guarantee with, and it lands on a four-person team.
- **Every customer must be on a version we still support.** Without a deployment pipeline that can upgrade them all, they drift apart and a bug fix has to be applied N times.

- **Infrastructure cost per customer has a floor.** Two databases and object storage cost something even at zero load, so a small customer may be unprofitable at a low price.
- **No cross-customer anything.** No aggregate benchmarking ("your time-to-fill versus similar companies"), no shared talent pool, no product analytics without a separate pipeline that exports from each instance. If a future product decision needs any of those, this decision is what has to change.
- **Moving to multi-tenancy later is a migration, not a refactor.** Data from N databases must be merged and a tenant identity introduced everywhere at once. We accept this rather than pay for it in advance.
- **ADR-001's gateway loses one of its two stated jobs.** It still terminates TLS, validates the session token, resolves the role and routes; it no longer resolves a workspace or enforces tenant isolation. This record, not that one, is where that change is stated.

Related

Related decisions

- ADR-001 — its Issue and Implications treated tenant isolation as a reason for a single gateway and as a cost of decomposition. This record removes that force; the gateway keeps its other reasons. The service named *Identity & Workspace Service* becomes the **Identity Service**.
- ADR-003 — documents no longer carry a `workspace_id`; `candidate_id` alone locates a person's derived data for erasure.
- ADR-005 *(Proposed)* — its highest-risk implication was workspace-identity enforcement implemented in several languages. That risk is removed by this decision, which makes per-service language choice meaningfully safer.
- ADR-004 — per-workspace token budgets become per-deployment budgets.
- **Still open:** the deployment and upgrade pipeline this decision makes necessary — how N instances are provisioned, configured and kept on a supported version.

Related requirements

- **FR-0.1** — the session identifies a user and a role, not a company.
- **Workspace scoping** — the requirement is withdrawn by this decision.
- **FR-5.1** — retention policy is per deployment.
- **NFR-12** — the access audit log remains, per deployment.
- **NFR-09** — availability is now per customer instance: a smaller blast radius, and N things to monitor.
- What is required of us is unaffected; the architecture is unchanged in shape.

Related artifacts

The sign-in and access-management use cases, the actor descriptions, the use case diagram, and the deployment configuration once implementation starts.

Related principles

- *A guarantee by construction beats a guarantee by discipline.*
- *Do not build plumbing for a scale you do not have* — the dormant tenant id was rejected on exactly this ground.
- *Pay operational cost to remove a correctness risk when the team cannot carry the risk.*

Notes

The workspace concept was never decided. It arrived with the multi-tenant framing and spread into the requirements and four ADRs unchallenged, surfacing only when the actors' actual capabilities were listed and nobody could say who creates a workspace — the first Admin appeared from nowhere. That is the second decision found hiding in an assumption, after the backend language.

The strongest objection is commercial rather than technical: single-tenant products are harder to sell cheaply, and the assumption that nobody needs cross-customer data is the one most likely to be overturned by a real customer conversation — aggregate benchmarking is exactly the kind of thing an SME would find valuable. If that happens, this is the record to revisit, and revisiting it means a migration.

ADR-007: gRPC on the AI Service boundary

Date: 2026-09-20 **Deciders:** Patiphon Puntusin, Thanabul Parodom, Thanwarat Korcharoenkiat, Rerngrit Jangsri

> *In the context of one internal boundary that carries the highest call volume in the system and the narrowest, most load-bearing contract, facing services that may each be written in a different language and a contract that changes whenever the model output does, we decided to make the AI Service speak gRPC while every other boundary stays REST, to achieve a compiled contract and generated clients on both sides instead of two hand-written JSON clients kept in step by discipline, accepting a second wire protocol, a code-generation step in every build, and the loss of a boundary anyone could debug with `curl`.*

Summary

Issue

One protocol everywhere was chosen to get the boundaries placed before anything else was spent. That has happened: the services, their data ownership and their collaborations are settled and have survived an audit. The question now is whether every boundary should keep the same protocol *because* they were drawn together, and one of them should not.

The AI Service boundary is not like the others. Three things separate it.

Volume. Every other internal call happens once per job opening, once per batch, or once per retention sweep. This one happens once per resume — two hundred times for one recruiter action, and it is the call the system's throughput target is measured against.

Contract shape. It is the narrowest interface in the system and the one that must not drift: *(candidate profile, criteria) → (score, must-have check, justification)*. Every part of that is load-bearing. A renamed field on one side produces a missing justification or a silently absent must-have result rather than an error, and a score without its justification is exactly the output the product exists to prevent.

Language exposure. Services may each choose their own language. This boundary is the one most likely to be crossed by two different ones — the service holding the model credential has the strongest pull toward Python, and the two services that call it need not follow. A hand-written JSON client in two languages is two chances to disagree about a field name, and nothing catches it until a resume scores wrong in production.

Underneath all three: the contract changes whenever the model output does, which is more often than anything else in the system changes.

Decision

The AI Service speaks gRPC. Every other boundary stays REST over HTTP/JSON.

- Its three operations are defined in a `.proto` as RPCs — criteria extraction from a description, scoring a profile against criteria, and generating interview questions. That file is the contract; clients and servers are generated from it in whatever language each service is written in, and no client for this boundary is written by hand.

- It stays internal. It is not exposed through the gateway and no browser reaches it, which is what makes a non-browser-friendly protocol acceptable here and nowhere else.
- Field numbers are never reused and fields are never removed, only deprecated, so an older client keeps working against a newer server through a deployment.
- Everything else — client to gateway, gateway to service, and every other service-to-service call — is unchanged. This record narrows one boundary, not the rule.

Status

Accepted

Group

Communication

Details

Assumptions

- The three model operations are stable in *shape* even though their content evolves: a profile and criteria in, a structured judgement out. If that shape turned out to change too, no contract mechanism would help.
- Every language a service might be written in has a maintained gRPC implementation. This is true of every mainstream candidate and is a gate on any unusual one.
- Call volume stays in the range of a few hundred per batch. gRPC is not being adopted for performance; the provider's own latency dominates anything the wire format contributes.
- Four people can carry one code-generation step in the build. Two would be a different decision.

Constraints

- A gRPC service is required of us, and no boundary currently provides one. This record closes half of a gap that was recorded as deliberate rather than accidental; the message-broker half remains open.
- The boundary this applies to, and the rule that every boundary was REST, were set by ADR-001. This record overrides that rule for this one boundary and leaves the rest of it standing.
- The three operations themselves, and the requirement that the service is never reachable from the gateway, were fixed by ADR-004 and are not reopened here.
- Whatever is chosen must work across more than one implementation language, because language is a per-service choice.

Positions

55. **Keep REST on every boundary**, holding the line set when the boundaries were drawn.
56. **gRPC on the AI Service boundary only** *(chosen)*.
57. **gRPC on every internal call**, REST only at the gateway.
58. **Keep REST, but generate the clients** from an OpenAPI document instead of writing them.
59. **GraphQL** for the internal calls.

Argument

Against holding the line (1). The reason for one protocol was that boundaries are expensive to get wrong and protocols are not — so place the boundaries first and pay for protocol variety later. That argument does not say *never*; it says *not yet*. The boundaries are now placed and audited, so the condition it was waiting on has been met. Keeping REST here would mean maintaining two hand-written JSON clients across a language boundary for the highest-volume, least forgiving contract in the system, which is precisely the case the deferral was meant to be revisited for.

Why this boundary and not the others (3). Erasure, the screening hand-off and the pipeline pushes are low-volume, their shapes change rarely, and two of them are how services tell each other work has happened rather than requests for a computed answer. Converting them buys little and costs a second toolchain on every path, including the callback into the service that started the work. The AI boundary earns the exception on all three counts the others fail: volume, contract fragility, and the likelihood of two languages meeting across it.

Against generated REST clients (4) — the closest alternative and the one that nearly won. It delivers the main benefit, generated clients, without a second wire protocol, and the discipline of writing the contract before the endpoint is already required of us. It was rejected because the document stays advisory: the generated client believes the document, the server can return something else, and the two only disagree at runtime. With protobuf the schema *is* the wire format, so a response that does not match the contract cannot be produced in the first place. On the boundary where a missing field silently becomes a missing justification, that difference is the whole point. Generator maturity across languages is also uneven in a way protobuf's is not.

Against GraphQL (5). It solves over-fetching across a graph of related resources, which is not this problem. This is one call with a fixed input and a fixed answer.

On "easier to consume", which is worth stating plainly because it is the reason people usually reach for the opposite conclusion. For a browser, an outside integrator, or anyone holding `curl`, REST is easier and gRPC is worse — no network tab, no readable payload, a toolchain before the first request. That cost is real and this boundary avoids it only because it has none of those consumers: it is internal, and it is never exposed through the gateway. *Inside* the system, "easy to consume" means a caller gets a typed client it did not write and cannot get wrong, which is what this buys. If the AI Service ever needs an external consumer, that consumer gets a REST façade in front of it and this decision is revisited.

Implications

What this buys us

- The contract is a compiled artifact. A field renamed on one side fails at build time in the other, instead of producing a plausible-looking score with no justification attached.
- Neither side writes a client. Adding a third caller later costs a generation step, not a new hand-rolled client and a new set of bugs.
- Two languages can meet on this boundary safely, which is what makes per-service language choice affordable rather than theoretical.
- Streaming is available if a long generated answer should arrive incrementally. Nothing needs it today; it stops being a future rewrite.

- One of the two communication styles still owed is now delivered.

What it costs us

- **A second wire protocol, and it lands on the team, not just the code.** Everyone who debugs scoring now needs a gRPC client to see a request. The browser network tab, which shows every other call in the system, shows nothing here.
- **A code-generation step in every build** for every language a caller is written in, plus the `.proto` itself needing a home and an owner. Where shared contract definitions live is an open question this decision makes urgent rather than answers.
- **The rule "every boundary is REST" is no longer true**, so anything stating it as a blanket fact is now wrong, and every diagram and table showing collaborations has two protocols to mark instead of one. A uniform rule is cheaper to hold than a rule with one exception, and the exception has to keep earning it.
- **The guardrail that every boundary is defined by an OpenAPI document needs a second arm** for the boundary that now has a `.proto` instead. One contract mechanism becomes two.
- **A local demo needs the generated stubs present**, so a checkout that has not run the generator does not build. That is a new way for a new team member's first day to go wrong.
- **It reverses part of a decision taken nine days ago.** Whatever the argument, a reader is entitled to ask whether the first decision was made too quickly, and the honest answer is in the Notes.

Related

Related decisions

- ADR-001 — set one protocol on every boundary and named this call as the first place to revisit if a boundary demonstrated it needed something else. **This record overrides that rule for the AI Service boundary only**; it stands everywhere else.
- ADR-004 — defined the three operations and made the service unreachable from the gateway. This record changes how they are carried, not what they are or who may call them.
- ADR-005 — per-service language choice is the force that makes a generated contract worth paying for, and its contract guardrail now covers two mechanisms.
- **Still open, and made more pressing:** where shared contract definitions live in the repository, and who owns the `.proto`.

Related requirements

- **Criteria extraction from a free-text description** (UC-1, the use case for creating a job opening), **scoring with a justification** (UC-2, batch screening) and **interview question generation** (UC-3) are the three operations this boundary carries. FR-1.2, FR-2.5 and FR-3.3 — the derived criteria, the score with its must-have check and written justification, and each question tagged with the criterion it tests — are the fields the contract must not lose.
- **NFR-17** — the model and prompt change inside one service; a versioned contract is what keeps that true for callers when the shape of the output shifts with it.
- **NFR-06, NFR-07, NFR-08** — screening throughput and its proportionality to worker count. This is the call on that path; the wire format is not the bottleneck, but a broken contract stops the measurement entirely.

- Required of us: a service reached by gRPC, which this provides. The service driven by a message broker is still not provided.

Related artifacts

The `.proto` defining these three RPCs, once written, and the generated clients in each calling service. Any diagram or table of collaborations now marks two protocols.

Related principles

- *Explainability is required, not optional* — the justification and the per-criterion evidence travel on this boundary, and a contract that can lose a field quietly is a contract that can lose an explanation quietly.
- *Easily reversible* — the three operations are domain-shaped, not model-shaped and not protocol-shaped. Putting REST back, or a façade in front, changes the transport and leaves the interface intact.

Notes

This narrows a decision taken nine days earlier, and that deserves an answer rather than a footnote. The earlier record did not claim REST everywhere was permanent; it argued that boundaries are expensive to move and protocols are not, so boundaries should be placed first and protocol variety bought afterwards, and it named this call as the first candidate. What has changed is that the boundaries are now settled. We are converting on the cross-language and contract-fragility argument rather than waiting to be bitten by drift in production, which the earlier record described as the trigger. Acting before the failure rather than after it is the intended use of a deferral, not a reversal of it.

The honest tension is the requirement. A gRPC service is expected of us and none existed, so this record is convenient as well as correct. What makes it defensible is that the boundary was not chosen to satisfy the requirement: it is the one the earlier record already identified, on properties — volume, contract fragility, language exposure — that would hold if the requirement did not exist. Had the requirement pointed at a different boundary, this record would still point here.

The message-broker half of that obligation remains open, and should not be closed the same way. The place it would earn its keep is already identified — the pipeline events a service currently pushes synchronously into the write path of another, where a dropped call leaves a permanently wrong number and nothing to reconcile against. That is a real problem looking for a broker, and it deserves its own record when it is taken.

ADR-008: Turning a resume into a candidate profile is a model call

Date: 2026-10-01 **Deciders:** Patiphon Puntusin, Thanabul Parodom, Thanwarat Korcharoenkiat, Rerngrit Jangsri

> *In the context of resumes whose layout is chosen by the candidate and cannot be specified in advance, facing a rule-based parser that must guess at a document it has not seen, we decided to extract the structured candidate profile with a language model, through the same service that already holds the model credential, to achieve a parser that degrades visibly instead of silently, accepting one more model call on the highest-volume path, a slower parsing bound, and the loss of a privacy guarantee that until now held by the shape of the contract.*

Summary

Issue

Screening a resume has two steps before a score exists. A PDF becomes text, and that text becomes a **candidate profile** — the normalised record of work history, skills and education that scoring is performed against.

The first step is genuinely deterministic and stays that way. The second was specified as deterministic too, with a 5-second bound, on the assumption that it is text processing. It is not.

The input is adversarial by nature, not by intent. A resume's layout is chosen by the person being evaluated, for aesthetic reasons, from a template we have never seen. We cannot specify it, cannot reject it, and cannot ask for a resubmission — the candidate is not our user and has no interface to the system. Every rule a parser applies is a guess about a document that has not arrived yet. Two columns, a table, a designer's layout, or a Thai resume dating its jobs in the Buddhist era, and the guess is wrong.

And wrong quietly. A rule-based parser that pairs a job title with the wrong employer does not fail. It emits a well-formed profile, which is scored with confidence, and produces a ranking with a justification that reads plausibly and is about a career the candidate does not have. That is not a parsing defect one layer down; it is the product's central failure — a candidate judged without anyone having actually read them — reintroduced in the one place nobody is looking.

The forces:

- **We have no leverage over the input format**, and no ability to obtain a corrected one.
- **A silent wrong profile is worse than a loud failure.** Marking a resume for manual review costs a recruiter two minutes; a plausible wrong score costs a candidate the role.
- **Structuring is what language models are good at**, and what regular expressions are worst at.
- **Thai and English mixed in one document** is normal in this market, and multiplies the number of rules a deterministic parser needs.
- **Every model call is paid for, and this is the highest-volume path in the system** — once per resume, two hundred times for one recruiter action.

- **Sending a whole resume to a model means sending the parts we deliberately exclude from scoring** — name, age, nationality, photograph.

Decision

The candidate profile is produced by a model call, served by the same service that holds the model credential, as a fourth operation alongside criteria extraction, scoring and interview questions.

- **Text extraction stays deterministic and stays where it is.** The service that owns resume files turns the PDF into text in process. No model sees a file.
- **The new operation takes resume text and a language, and returns a candidate profile.** It is the only operation that receives unstructured resume text.
- **The profile is extracted once per resume and stored**, in the document store that already holds candidate profiles. Scoring is once per resume per job opening; extraction is not repeated when a second opening is scored, when a batch is re-run, or when the talent pool is re-matched.
- **Protected attributes are dropped during extraction.** Gender, age, marital status, religion, nationality and photograph are not fields of a candidate profile, so the extraction operation is instructed not to carry them into one, and nothing downstream can reintroduce them.
- **A resume the model cannot structure is marked for manual review**, under the same retry-then-abandon path that already governs a failed score. It is never given an empty profile.

The parsing bound moves from 5 seconds to 20 seconds at the 95th percentile, with a 60-second abandon — the same figures scoring already carries, because it is now the same kind of work.

Status

Accepted

Group

Data · Integration · Privacy

Details

Assumptions

- A language model structures a resume more reliably than a rule set we could write and maintain in the time available. *This is the assumption the decision rests on, and it has not been measured — see Notes.*
- Extraction output is small and mostly input tokens, so its cost per resume is below scoring's.
- Resume text extracted from a PDF is faithful enough to structure. Scanned images are out of scope; those need character recognition, which is a separate decision.
- The document store can hold profiles long enough to make caching worthwhile, within the retention policy that already governs candidate data.

Constraints

- Four part-time developers on a deadline, none of whom has written a production resume parser.
- Personal data law makes what is sent to a third-party processor a compliance question, not only a cost one.

- The model provider's rate limit, not our hardware, bounds throughput on this path.

Positions

60. **Rule-based structuring** — layout-aware extraction with coordinates and font weights, section segmentation by heading keywords, a skills dictionary, and date regular expressions.
61. **Model-based structuring through the service holding the credential.** *(Selected.)*
62. **Send the resume file itself to the model**, and let it do extraction and structuring in one call.
63. **Structure nothing.** Put the whole resume text in the profile's free-text field and let scoring read it unstructured.
64. **Rule-based first, model only on the residue** — parse what matches confidently, ask the model about the rest.

Argument

Against rule-based structuring (1). Not that it cannot be built — parts of it work well. A skills dictionary is genuinely reliable, because membership of a known list is not inference. Dates yield to enough regular expressions. What does not yield is the association: deciding which title belongs to which employer over which period, which is layout-dependent and therefore unbounded. That is also the part scoring depends on most, since every criterion about experience is answered from it. The decisive objection is the failure mode, not the accuracy: a rule set that mis-associates produces a confident, plausible, wrong profile, and nothing in the system can tell that from a right one.

Against sending the file (3). It saves a step and costs the boundary. The service holding the credential would take an arbitrary binary from a caller, and the rule that it receives only what a judgement needs — which is what keeps the cost and the exposure of the model path reviewable in one place — would no longer be true of it. Text extraction is the one part of this pipeline that *is* deterministic, and there is no reason to buy it from a model.

Against structuring nothing (4). Attractive, and nearly viable: scoring already reads the profile's free-text field, so an unstructured resume would still be scored. It fails on two counts. The recruiter-facing views need structure — filtering a shortlist by criterion, showing years of experience, recognising a candidate who has applied before — and none of that is possible over a blob. More seriously, it destroys the privacy property described below: a raw dump carries every excluded attribute into every scoring prompt, on every resume, forever.

Against the hybrid (5). The best idea here and the wrong one for now. It needs the rule set *and* the model path *and* a confidence measure to decide between them — three things to build and tune where we have time for one. It is the natural optimisation once there is a measurement to optimise against, and it is worth revisiting when the cost per batch is known.

For model-based structuring (2). It puts the judgement where judgement is possible, and it puts it behind the boundary that already exists for exactly this kind of work — one credential, one place where prompts live, one place where cost is metered, and no new component. Caching makes the cost bearable: extraction is per resume, not per resume per opening, so the expensive case — a talent pool re-matched against many openings — pays for it once. And it turns the privacy handling from something a rule set would have to do by hand into a single instructed step in one prompt.

Implications

What this buys us

- The profile is produced by the thing best suited to producing it, on documents whose shape we cannot predict.
- A resume that cannot be structured is *marked for manual review*, not silently mis-structured. The failure becomes visible, which is the property the product is built around.
- Thai and English need no separate rule sets.
- Extraction is the one place where excluded attributes can be dropped deliberately, rather than by hoping no rule accidentally captures a photograph caption.
- A stored structured profile is what re-matching a talent pool against a new opening requires, so this decision pays for a capability that is currently deferred.

What it costs us

- **One more model call on the highest-volume path.** Amortised over re-scoring, but the first screening of a batch now makes two calls per resume instead of one. A runaway batch costs twice as much as it did.
- **Parsing is no longer fast.** The bound quadruples, from 5 seconds to 20. Perceived batch latency rises, and the throughput target is now bounded by the provider's rate limit on both steps rather than one.
- **A privacy guarantee weakens from structural to instructed.** Until now, excluded attributes could not reach a prompt because the profile has no field to carry them — a property of the contract's shape, which no one could forget. Extraction sees the whole resume, so that guarantee now rests on the extraction prompt behaving, and on a test. This is a real reduction and is accepted deliberately: the alternative is a rule set that must achieve the same thing by hand, in more places, with no better assurance.
- **An argument for the current service split weakens.** The two services were separated partly because their work has opposite resource profiles — one processor-bound and quick, the other waiting on a remote call. Both are now waiting on a remote call. The split stands on its other reasons: a model outage must not reach sign-in or the dashboard, and only one service holds the credential. That argument, not the resource one, is now what carries it.
- **Extraction quality is unmeasured.** No resume has been through either path. The decision is made on the shape of the problem rather than on evidence.
- **Scanned resumes remain unhandled**, and now look like a gap in a model-based path rather than a known limit of a text extractor.

Related

Related decisions

- **ADR-001** drew the service boundaries, and argued for separating resume processing from model access partly on their opposite resource profiles. That specific argument no longer holds; the boundary stands on failure isolation and credential containment.
- **ADR-002** defines one independently retried unit of work per resume, with a bounded retry and a terminal *needs manual review* state. Extraction failure uses that path unchanged; the unit of work now contains two model calls rather than one.

- **ADR-003** places AI-derived documents in the document store and the system of record in the relational database. The extracted profile is an AI-derived document and goes where that record already says, so caching it introduces no new storage decision.
- **ADR-004** requires that the service holding the credential receive only what a judgement needs, and never a raw file. The file rule is unchanged. The minimisation rule is narrowed: this one operation receives unstructured resume text, and is the reason the excluded attributes are dropped there rather than assumed absent.
- **ADR-005** *(Proposed)* prefers Python for resume processing partly because of its PDF-and-text libraries. Text extraction still happens there, so the preference stands, on a smaller argument.
- **ADR-007** carries the model service's operations over a compiled contract. The new operation is defined in the same contract and generated the same way.
- **Still open:** whether a rule-based pre-pass in front of the model is worth building once cost per batch is measured; and character recognition for scanned resumes.

Related requirements

- **FR-2.3** — a resume is parsed into a normalised candidate profile. Unchanged in what it requires; this record decides how.
- **FR-2.7** — retry failed attempts and flag what cannot be parsed or scored for manual review. Now covers a failed extraction as well as a failed score.
- **NFR-01** — parsing within 20 seconds at the 95th percentile, abandoning at 60. Raised from 5 seconds by this decision.
- **NFR-13, NFR-14** — erasure, and the exclusion of protected attributes from scoring. NFR-14 is now enforced by instruction and test at extraction rather than by the profile's shape alone.
- **NFR-16** — Thai and English. The reason one path now serves both.
- **NFR-17** — the scoring model and prompt change within one service boundary. Extraction is now inside that same boundary and gains the same property.

Related artifacts

The shared contract definition for the model service, the runtime description of batch screening, and the per-service operations table, each of which currently states that profile extraction is not a model call.

Related principles

- *Prefer a loud failure to a plausible one.*
- *Do not write rules for input you do not control.*
- *Pay once for what is reused* — extraction is per resume, scoring is per resume per opening.
- *A guarantee by construction beats a guarantee by discipline* — and when one is traded for the other, say so rather than letting it lapse quietly.

Notes

The assumption underneath this record is unmeasured: no resume has been run through either path. It was taken on the structure of the problem — an input we do not control, and a failure that does not announce itself — rather than on a comparison, because building the rule-based parser well enough to compare fairly would cost most of the time the decision is trying to save.

The honest counter-argument is cost. Doubling calls on the busiest path is the kind of decision that looks obvious in design and expensive in an invoice, and the hybrid in position 5 is the answer if it does. Extraction caching is what makes the difference survivable, and it is the first thing to verify actually works.

The privacy trade is the part worth returning to. Swapping a property the contract enforced for one a prompt is asked to observe is a real loss, taken because the alternative offered no better assurance and more places to get it wrong. If a rule-based pre-pass is ever built, restoring that guarantee should be one of the things it buys back.